

Cours 2: Data Set

Luis Alberto Gomez

Master 1 SE et DS2E

Année 2025 – 2026

Plan de la séance

- 1 Création d'une base de données vide
- 2 Création avec DATA et INPUT
- 3 Lecture depuis un fichier externe avec INFILE
- 4 Créer un Data Set à partir d'un autre avec SET
- 5 Édition d'une base de données
 - Fonctions pour les variables caractères
 - Fonctions mathématiques
 - Sélection ou suppression de variables
 - Filtrer les observations
- 6 Importation et exportation de fichiers
 - Importer des fichiers externes
 - Exporter vers un fichier externe

Créer une base de données vide

```
data base_vide;  
run;
```

Explications :

- `data base_vide;` : crée un Data Set vide nommé `base_vide`.
- Aucune variable n'est définie, donc la table est complètement vide.
- `run;` : exécute le code pour créer la base.

Création d'un Data Set avec DATA et INPUT

```
data etudiants;  
    input nom $ prenom $ age note;  
    datalines;  
Dupont Jean 20 15  
Martin Alice 21 17  
Durand Paul 22 14  
;  
run;
```

Explications :

- `data etudiants` : crée un Data Set nommé `etudiants`.
- `input` : définit les variables et leurs types (\$ pour les caractères).
- `datalines` : permet de saisir directement les données.
- **Remarque** : par défaut, les valeurs dans `datalines` sont séparées par un espace.

Troncature des variables caractères après 8 caractères

Par défaut, si une variable caractère dépasse 8 caractères, SAS tronque la valeur à 8 caractères.

```
DATA students;  
    INPUT name $ gender $ age;  
    DATALINES;  
David Male 19  
Amelia Female 23  
PierrePolJacques Male 28  
;  
RUN;
```

Explications :

- La variable name est de type caractère.
- Les noms dépassant 8 caractères seront tronqués automatiquement par SAS. Par exemple, PierrePolJacques devient PierrePo.

Astuce : changer la longueur d'une variable caractère

Pour éviter la troncature des variables caractères (par défaut 8 caractères), il faut définir la longueur maximale avant INPUT avec length :

```
DATA students;  
    length name $20;  
    INPUT name $ gender $ age;  
    DATALINES;  
PierrePolJacques Male 28  
Amelia           Female 23  
David            Male 19  
;  
RUN;
```

Explications :

- length name \$20; : définit la variable name avec une taille maximale de 20 caractères.

Différence entre **FORMAT** et **INFORMAT** :

- **INFORMAT** : indique à SAS comment lire les valeurs des observations associées à une variable.
- **FORMAT** : indique à SAS comment afficher les valeurs dans les tables ou les sorties.

Définir un **INFORMAT** selon le type de variable :

- Variables numériques : **w.d**
 - **w** = largeur totale de la valeur (chiffres + point décimal)
 - **d** = nombre de chiffres après la virgule
- Variables caractères : **\$w**
 - **w** = largeur maximale de la chaîne

Exemple pratique :

```
data clients;  
    input nom $20 age 3.2 salaire 6.2;  
    format nom $20.  
           age 3.  
           salaire 6.2;  
    datalines;  
Dupont 25 2500.50  
Martin 30 3200.75  
;  
run;
```

Explications :

- nom \$20 : informat pour lire jusqu'à 20 caractères.
- age 3.2 : informat numérique (largeur 3, 2 décimales).
- La partie format définit l'affichage de ces variables dans SAS.

Important à savoir lors de l'utilisation des INFORMAT :

- SAS s'arrête de lire une valeur dès qu'il rencontre un espace et passe à la variable suivante.
- Il est important d'assigner un **INFORMAT** aux variables pour que SAS lise correctement les valeurs, surtout pour les dates ou les variables avec des formats particuliers.

Remarque : L'informat garantit que SAS interprète correctement la valeur, même si elle contient des espaces ou des caractères spéciaux.

Importer un fichier texte ou CSV

La commande INFILE permet de spécifier l'URL ou le chemin vers un fichier de données brut. Il s'agit généralement d'un fichier .txt ou .csv.

```
data ventes;  
    infile 'C:\MesFichiers\ventes.csv';  
    input produit $ quantite prix;  
run;
```

Explications :

- infile : chemin du fichier texte ou CSV.
- input : liste des variables à lire.

Introduction : La commande FILENAME permet de créer un alias pour un fichier externe (texte ou CSV) que SAS pourra lire ensuite. L'option dlm permet de spécifier le caractère délimiteur entre les valeurs dans le fichier.

```
filename ventes 'C:\MesFichiers\ventes.csv';  
  
data ventes_data;  
    infile ventes dlm=', ' ;  
    input produit $ quantite prix;  
run;
```

Explications :

- filename : crée un alias ventes pour le fichier CSV.
- infile ventes : utilise l'alias pour lire le fichier.
- dlm=', ' : indique que les valeurs sont séparées par une virgule.

Options FIRSTOBS et OBS pour lire un fichier

Introduction : Les options FIRSTOBS et OBS permettent de contrôler quelles lignes d'un fichier texte ou CSV sont lues par SAS.

```
data ventes_subset;  
    infile 'C:\MesFichiers\ventes.csv' dlm=','  
        firstobs=2 obs=10;  
    input produit $ quantite prix;  
run;
```

Explications :

- firstobs=2 : SAS commence la lecture à la ligne 2, utile pour ignorer l'en-tête.
- obs=10 : SAS s'arrête à la ligne 10, même si le fichier contient plus de lignes.
- Combinées, ces options permettent de lire seulement un sous-ensemble du fichier.

Créer un nouveau Data Set à partir d'un existant

```
data etudiants_maj;  
  set etudiants;  
  age = age + 1; /* met a jour l'age */  
run;
```

Explications :

- `set etudiants` : lit chaque observation du Data Set `etudiants`.
- Modification possible des variables existantes ou création de nouvelles variables.

Fonctions pour les variables caractères (1)

Fonctions utiles pour manipuler des chaînes de caractères :

- **Concaténation** : l'opérateur `||` permet de concaténer des chaînes.

```
1 fullname = nom || prenom;
```

- `lowercase()` : convertit toutes les lettres en minuscules.

```
1 nom_min = lowercase(nom);
```

- `uppercase()` : convertit toutes les lettres en majuscules.

```
1 nom_maj = uppercase(nom);
```

- `char(string,n)` : retourne le caractère à la position `n` dans une chaîne.

```
1 lettre = char(nom,2);
```

Fonctions pour les variables caractères (2)

- `index(string,recherche)` : recherche une expression dans une chaîne et retourne la position de la première occurrence.

```
1 pos = index(nom, 'a');
```

- `compress(chaine,caract)` : supprime certains caractères d'une chaîne.

```
1 nom_sans_espace = compress(nom, ' ');
```

Remarque :

- `compress(chaine, ' ')` supprime tous les espaces dans une chaîne.
- On peut aussi supprimer plusieurs caractères en même temps.

Opérateurs et fonctions mathématiques courantes :

- **Opérateurs usuels** : +, -, *, /
- **abs()** : retourne la valeur absolue.

```
1 val_abs = abs(x);
```

- **exp()** : retourne l'exponentielle.

```
1 exp_val = exp(x);
```

- **log()** : retourne le logarithme naturel.

```
1 log_val = log(x);
```

- **sqrt()** : retourne la racine carrée.

```
1 racine = sqrt(x);
```

- **round()** : retourne l'entier le plus proche.

```
1 val_arrondie = round(x);
```

```
data etudiants_sel;  
    set etudiants(keep=nom note);  
run;  
  
data etudiants_sans_age;  
    set etudiants(drop=age);  
run;
```

Explications :

- keep : liste des variables à conserver.
- drop : liste des variables à supprimer.

```
data etudiants_bons;  
  set etudiants;  
  where note >= 15;  
run;
```

Explications :

- **where** : sélectionne uniquement les observations qui remplissent la condition.

```
proc import datafile="C:\MesFichiers\ventes.csv"  
    out=ventes  
    dbms=csv  
    replace;  
    getnames=yes;  
run;
```

Explications :

- dbms=csv : type de fichier.
- replace : remplace le fichier existant si déjà présent.
- getnames=yes : utilise la première ligne comme noms des variables.

```
proc export data=etudiants  
    outfile="C:\MesFichiers\etudiants.csv"  
    dbms=csv  
    replace;  
run;
```

Explications :

- Exporte le Data Set vers un fichier CSV.
- replace : écrase le fichier existant.